

南开大学

信息隐藏技术课程实验报告

实验六：LSB 隐藏法



学 院 密码与网络空间安全学院
专 业 信息安全、法学双学位班
学 号 2313815
姓 名 段俊宇
班 级 信息安全、法学双学位班

一、实验要求

- 1、实现将二值位图嵌入到图像中。
- 2、实现将学号（一个整数以及南开校徽二值图）嵌入到位图中。

二、实验原理

1. LSB(Least Significant Bit 最不重要位/最低有效位)

基本原理：用秘密信息(比特)替换掉最低有效位的数据。

符号约定如下：

- C_i ：载体对象的取样值。
- $L(c)$ ：载体的长度。
- $L(m)$ ：秘密信息的长度。
- S_i ：伪装对象的取样值。
- k ：伪装密钥。

流载体的 LSB 方法：

- 嵌入：选择一个载体元素的子集 $j_1, j_2, \dots, j_{L(m)}$ ，其中共有 $L(m)$ 个元素，用以隐藏秘密信息的 $L(m)$ 个比特。然后在这个子集上执行替换操作，把 C_{j_i} 的最低比特用 m_i 来替换。
- 提取：找到嵌入信息的伪装元素的子集 $j_1, j_2, \dots, j_{L(m)}$ ，从这些伪装对象 S_{j_i} 中抽出它们的最低比特位，排列之后组成秘密信息。

选择隐藏位置子集的方法：

- 顺序选取：发送者从载体的第一个元素开始，顺序选取 $L(m)$ 个元素作为隐藏的子集。
 - 缺点：载体的已修改部分和未修改部分，具有不同的统计特性。
 - 解决方法：
 1. 嵌入秘密信息后，继续嵌入伪随机序列。
 2. 重复嵌入秘密信息，直到载体结束。
 3. 随机间隔法：双方利用相同的伪随机数发生器和伪装密钥，确定隐藏位置。

- 随机间隔法：用伪装密钥 k 作为伪随机数发生器的种子，生成一个伪随机序列 $k_1, k_2, \dots, k_{L(m)}$ ，嵌入位置为：

$$j_1 = k_1$$

$$j_i = j_{i-1} + k_i \quad i \geq 2$$

伪随机置换：能够得到载体的所有元素(例如：载体为图像)，就可以从整个载体考虑，把秘密信息比特随机地分散在整个载体中。

- 嵌入：用种子密钥产生一个伪随机索引序 $j_1, j_2, \dots, j_{L(m)}$ ，将第 k 个秘密消息比特隐藏在索引为 j_k 的载体元素的最低比特位中。
- 提取：用同样的种子密钥产生同样的伪随机索引序列，从这些位置最低比特位中提取信息。
- 应避免的问题：产生的索引值可能产生碰撞；用一个集合记录所有已使用过的载体索引值，当再次出现同样的索引值时，则放弃这个索引值，再选择下一个元素；提取时做同样处理。

LSB 方法的特点：

- 优点：简单、易实现、容量大。
- 缺点：安全性不高、不能抵抗叠加噪声、有损压缩等破坏。

提高 LSB 方法的安全性的措施：

1. 对秘密信息先加密后再隐藏。
2. 多次重复嵌入。
3. 引入纠错编码技术。先进行纠错编码，再进行隐藏。

三、实验环境

Matlab_R2025b, Image Processing Toolbox

四、实验过程

1. 学号的嵌入

如下所示，我将使用注释的方式对代码进行解释。

```
1. % 信息隐藏函数
2. function hide_message(input_image, secret_message, output_image, secret_key,
position_file)
3.     img = imread(input_image);
```

```

4.     [h, w, c] = size(img);
5.
6.     % 将字符串转为 UTF8 字节, 兼容中文
7.     message_bytes = unicode2native(secret_message, 'UTF-8');
8.     message_length = length(message_bytes);
9.     length_bits = dec2bin(message_length, 32) - '0';
10.    message_bits = reshape(dec2bin(message_bytes, 8).' - '0', 1, []);
11.    data_bits = [length_bits message_bits];
12.
13.    % 图像容量
14.    capacity = h * w * c;
15.    if length(data_bits) > capacity
16.        error('图像容量不足');
17.    end
18.
19.    % 转为一维
20.    img_vec = img(:);
21.    total_pixels = length(img_vec);
22.
23.    % 使用密钥生成随机位置
24.    rng(secret_key);
25.    all_positions = randperm(total_pixels);
26.    positions = all_positions(1:length(data_bits));
27.
28.    % 保存位置信息
29.    save(position_file, 'positions');
30.
31.    % LSB 嵌入到随机位置
32.    for i = 1:length(data_bits)
33.        img_vec(positions(i)) = bitset(img_vec(positions(i)), 1, data_bits(i));
34.    end
35.
36.    % 恢复图像
37.    stego_img = reshape(img_vec, h, w, c);
38.    imwrite(stego_img, output_image);
39.
40.    % 计算 PSNR
41.    [mse, psnr] = calculate_psnr(img, stego_img);
42.    fprintf('隐写完成\n');
43.    fprintf('PSNR: %.2f dB\n', psnr);
44. end
45.
46. % 信息提取函数
47. function message = extract_message(stego_image, position_file)

```

```

48.     fprintf('开始提取信息...\n');
49.
50.     % 加载位置信息
51.     load(position_file, 'positions');
52.
53.     img = imread(stego_image);
54.     img_vec = img(:);
55.
56.     % 读取 32bit 长度
57.     length_bits = zeros(1, 32);
58.     for i = 1:32
59.         length_bits(i) = bitget(img_vec(positions(i)), 1);
60.     end
61.     message_length = bin2dec(char(length_bits + '0'));
62.
63.     % 读取消息 bit
64.     total_bits = message_length * 8;
65.     message_bits = zeros(1, total_bits);
66.     for i = 1:total_bits
67.         message_bits(i) = bitget(img_vec(positions(i+32)), 1);
68.     end
69.
70.     % 将 bit 转换为 byte
71.     message_bytes = zeros(1, message_length);
72.     for i = 1:message_length
73.         start_idx = (i-1) * 8 + 1;
74.         end_idx = i * 8;
75.         byte = message_bits(start_idx:end_idx);
76.         message_bytes(i) = bin2dec(char(byte + '0'));
77.     end
78.
79.     % 将 UTF8 转换为字符串
80.     message = native2unicode(uint8(message_bytes), 'UTF-8');
81.     fprintf('提取完成\n');
82. end

```

这部分代码是对文本内容的嵌入和提取。嵌入时首先将文本内容转为 UTF-8 编码以兼容中文，接着将图像转为一维向量，使用密钥生成随机位置并保存位置文件，从而实现信息的隐藏。提取时通过读取位置信息提取隐藏的比特，然后将其转换为字符串，实现对隐藏信息的提取。

如下所示是计算 PSNR 的函数：

```

1. % PSNR 计算
2. function [mse, psnr] = calculate_psnr(original, stego)

```

```

3.     original = double(original);
4.     stego = double(stego);
5.     mse = mean((original(:) - stego(:)).^2);
6.     if mse == 0
7.         psnr = Inf;
8.     else
9.         psnr = 10 * log10(255^2 / mse);
10.    end
11. end

```

2. 二值图像的隐藏

如下所示，我将使用注释的方式对代码进行解释。

```

1. % 图像嵌入函数
2. function hide_binary_image(input_image, binary_img_path, output_image, secret_key,
position_file)
3.     % 读取原始图像
4.     img = imread(input_image);
5.     [h, w, c] = size(img);
6.
7.     % 读取二值图像
8.     binary_img = imread(binary_img_path);
9.     % 确保是二值图像
10.    if ~islogical(binary_img)
11.        binary_img = im2bw(binary_img);
12.    end
13.    binary_img = double(binary_img);
14.
15.    % 获取二值图像的尺寸
16.    [bh, bw] = size(binary_img);
17.
18.    % 将二值图像转为二进制数据
19.    binary_data = binary_img(:)';
20.
21.    % 存储图像尺寸信息
22.    height_bits = dec2bin(bh, 16) - '0';
23.    width_bits = dec2bin(bw, 16) - '0';
24.
25.    % 拼接：尺寸信息 + 图像数据
26.    data_bits = [height_bits width_bits binary_data];
27.
28.    % 图像容量检查
29.    img_vec = img(:);
30.    total_pixels_img = length(img_vec);
31.    capacity = total_pixels_img;

```

```
32.     if length(data_bits) > capacity
33.         error('图像容量不足，无法嵌入二值图像');
34.     end
35.
36.     % 使用密钥生成随机位置
37.     rng(secret_key);
38.     all_positions = randperm(total_pixels_img);
39.     positions = all_positions(1:length(data_bits));
40.
41.     % 保存位置信息
42.     save(position_file, 'positions');
43.
44.     % LSB 嵌入
45.     for i = 1:length(data_bits)
46.         img_vec(positions(i)) = bitset(img_vec(positions(i)), 1, data_bits(i));
47.     end
48.
49.     % 恢复图像
50.     stego_img = reshape(img_vec, h, w, c);
51.     imwrite(stego_img, output_image);
52.
53.     % 计算 PSNR
54.     [mse, psnr] = calculate_psnr(img, stego_img);
55.     fprintf('二值图像隐写完成\n');
56.     fprintf('二值图像尺寸: %d x %d, 嵌入比特数: %d\n', bh, bw, length(binary_data));
57.     fprintf('PSNR: %.2f dB\n', psnr);
58. end
59.
60. % 图像提取函数
61. function binary_img = extract_binary_image(stego_image, position_file)
62.     fprintf('开始提取二值图像...\n');
63.
64.     % 加载位置信息
65.     load(position_file, 'positions');
66.
67.     img = imread(stego_image);
68.     img_vec = img(:);
69.
70.     % 读取尺寸信息
71.     height_bits = zeros(1, 16);
72.     width_bits = zeros(1, 16);
73.     for i = 1:16
74.         height_bits(i) = bitget(img_vec(positions(i)), 1);
75.     end
```

```

76.     for i = 1:16
77.         width_bits(i) = bitget(img_vec(positions(i+16)), 1);
78.     end
79.
80.     bh = bin2dec(char(height_bits + '0'));
81.     bw = bin2dec(char(width_bits + '0'));
82.     total_pixels = bh * bw;
83.
84.     % 读取图像数据
85.     binary_data = zeros(1, total_pixels);
86.     for i = 1:total_pixels
87.         binary_data(i) = bitget(img_vec(positions(i+32)), 1);
88.     end
89.
90.     % 恢复二值图像
91.     binary_img = reshape(binary_data, bh, bw);
92.     binary_img = logical(binary_img); % 转为 logical 类型
93.
94.     fprintf('提取完成, 二值图像尺寸: %d x %d\n', bh, bw);
95. end

```

这部分代码实现了对图像的隐藏和提取。隐藏时先将彩色图像转为二值图像并获取图像的基本信息，接着将图像转为二进制数据，使用密钥生成随机位置并保存位置文件，从而实现二值图像的隐藏。提取时先加载位置信息，并根据位置信息读取隐藏的比特，最后恢复原始图像。

3. 测试功能

如下所示，我将使用注释的方式对代码进行解释。

```

1. clc;
2. clear;
3. % 输入参数
4. input_image = '../Picture/1.bmp';
5. output_image = '../Picture/stego__number_image.bmp';
6. position_file = '../Picture/number_positions.mat'; % 保存位置信息
7. binary_image_path = '../Picture/2.bmp';
8. output_image_binary = '../Picture/stego_logo_image.bmp';
9. position_file_binary = '../Picture/logo_positions.mat';
10. secret_message = '2313815';
11. secret_key = 2026;
12.
13. % 隐藏信息
14. hide_message(input_image, secret_message, output_image, secret_key, position_file);

```



```
15. hide_binary_image(input_image, binary_image_path, output_image_binary, secret_key+1,  
position_file_binary);  
16. % 提取信息  
17. extracted = extract_message(output_image, position_file);  
18. fprintf('原始信息: %s\n', secret_message);  
19. fprintf('提取信息: %s\n', extracted);  
20. if strcmp(secret_message, extracted)  
21.     fprintf('✓ 验证成功: 信息完全一致\n');  
22. else  
23.     fprintf('✗ 验证失败\n');  
24. end  
25. extracted_binary = extract_binary_image(output_image_binary, position_file_binary);  
26. imwrite(extracted_binary, '../Picture/extracted_logo.bmp');  
27. fprintf('二值图像提取完成, 保存为 extracted_logo.bmp\n');
```

这部分代码实现了对上述隐藏和提取函数功能的测试，首先隐藏学号和南开校徽的图片，接着提取隐藏的文本消息和图片，并将提取的图片保存到本地进行比对。

下面是本次实验的原始图片载体：



原始的校徽图片如下所示：



接下来运行测试代码，结果如下所示：

```
隐写完成
PSNR: 101.16 dB
二值图像隐写完成
二值图像尺寸: 1031 x 1038, 嵌入比特数: 1070178
PSNR: 60.30 dB
开始提取信息...
提取完成
原始信息: 2313815
提取信息: 2313815
✓ 验证成功: 信息完全一致
开始提取二值图像...
提取完成, 二值图像尺寸: 1031 x 1038
二值图像提取完成, 保存为 extracted_logo.bmp
>>
```

从上面的结果可以看出，成功隐藏并提取了学号和二值图像。下面是提取的校徽的二值图像。



从上面的结果可以看出，原始的校徽图片和提取的校徽图片除了色彩之外没有任何的区别，说明成功实现了隐藏和提取。此外，用于隐藏的原始载体在隐藏后与隐藏前没有任何的区别，说明实验成功！

五、实验结论及心得体会

本次实验中我使用 LSB 方法对文本信息和图像实现了隐藏。在实验过程中，我体会到了 LSB 隐藏简单、容易实现和容量大的特性，同时为了更加安全，我使用伪随机置换方法实现，使用秘钥 2026 初始化随机数生成器并生成 1 到总像素数的随机排列，这样比简单的顺序嵌入更安全。